Лабораторная работа №1: Объектное моделирование предметной области Avtobys. Инкапсуляция и защита инвариантов

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Неделя: 1 из 10
Формат: Лабораторная работа, 3 академических часа (135 минут)
Среда выполнения: Python 3.12+, pytest, Git, VS Code / PyCharm

---

1. Цель и контекст работы

Цель работы: Применить принципы объектно-ориентированной декомпозиции и инкапсуляции для проектирования ядра доменной модели городской транспортной системы (Avtobys / Алматы-Транзит).
В этой работе вы перейдете от процедурного кода со словарями к строгой объектной модели с защитой инвариантов предметной области, инкапсуляцией состояния и юнит-тестами.

---

2. Требования к сдаче и структура репозитория

Все задания выполняются в персональном GitHub-репозитории студента: oop-kaznu-2026-<student_login>.
Структура папки для Лабораторной работы 1:

```
lab01_transit_domain/
├── domain/
│   ├── __init__.py
│   ├── bus.py          # Сущность Автобус (госномер, вместимость, текущий маршрут)
│   ├── route.py        # Сущность Маршрут (номер, список остановок, базовый тариф)
│   ├── card.py         # Транспортная карта с защитой баланса
│   └── ticket.py       # Электронный билет на поездку
├── tests/
│   ├── __init__.py
│   └── test_domain.py  # Набор тестов pytest для проверки защиты инвариантов
└── README.md           # Отчет: архитектурные решения, список инвариантов
```

Критерии оценки (20 баллов):

Блок	Задание	Баллы
Блок 1	Моделирование сущностей Bus и Route с валидацией	5

Блок	Задание	Баллы
Блок 2	Моделирование TransitCard и SingleTripTicket с защитой инвариантов	7
Блок 3	Разработка юнит-тестов (pytest) на проверку инвариантов и исключений	5
Блок 4	Защита работы: ответы на контрольные вопросы и демонстрация кода	3

---

3. Практические задания

Блок 1. Моделирование Bus и Route (5 баллов)

В файлах domain/bus.py и domain/route.py спроектируйте классы автобуса и маршрута.

Инварианты сущности Route:

1. route_number: строка, непустая (например, "32", "79", "12").
2. base_fare: базовая стоимость проезда в тенге (Decimal), строго положительное число (по умолчанию 100.00 KZT).
3. stops: упорядоченный список названий остановок (минимум 2 остановки: начальная и конечная). Названия остановок не могут дублироваться подряд.

Инварианты сущности Bus:

1. plate_number: государственный регистрационный номер (например, "123 AB 02"). Должен соответствовать формату (не пустой, без лишних пробелов).
2. capacity: максимальная пассажировместимость (целое число, от 10 до 250 человек).
3. current_passengers: текущее количество пассажиров в салоне (целое число, в диапазоне [0; capacity]).
4. Автобус может быть привязан к маршруту assign_route(route: Route) или снят с рейса.

```
# Пример обязательных методов Bus:
def board_passengers(self, count: int) -> None:
    """Посадка пассажиров. Выбрасывает CapacityExceededError, если превышена вместимость."""

def leave_passengers(self, count: int) -> None:
    """Высадка пассажиров. Выбрасывает ValueError, если число пассажиров становится < 0."""
```

Блок 2. Транспортная карта и Электронный билет (7 баллов)

В файлах domain/card.py и domain/ticket.py спроектируйте богатую объектную модель оплаты.

Бизнес-правила и инварианты:

1. **TransitCard**:
    - Баланс balance типа Decimal. Баланс никогда не может стать меньше 0.00.

- Поддержка льготных тарифов через коэффициент скидки discount_rate (например, 0.5 для студентов, 1.0 для бесплатных проездных).
- Методы:
  - top_up(amount: Decimal) — пополнение (минимум 100 KZT, только для незаблокированных карт).
  - pay(fare: Decimal) -> Decimal — списание. При нехватке выбрасывает InsufficientFundsError.
  - block(reason: str) и unblock() — управление блокировкой.

2. **SingleTripTicket**:
   - Разовый билет, приобретаемый за наличные или по SMS/QR без карты.
   - Имеет ticket_code (UUID), price, метку времени покупки issued_at и статус is_validated: bool.
   - Метод validate(bus: Bus) -> None:
     - Одноразовый: повторная валидация в том же или другом автобусе выбрасывает TicketAlreadyUsedError.
     - Срок действия: действителен только в течение 60 минут с момента покупки. При просрочке выбрасывает TicketExpiredError.

Блок 3. Автоматическое тестирование инвариантов (pytest) (5 баллов)

Напишите в tests/test_domain.py полноценный тестовый сьют, доказывающий надежность вашей инкапсуляции.
Обязательные тест-кейсы: 1. test_card_prevents_negative_balance: попытка списать сумму, превышающую баланс, вызывает InsufficientFundsError, а баланс карты остается неизменным. 2. test_card_blocked_rejects_operations: заблокированная карта отвергает и списание, и пополнение с исключением CardBlockedError. 3. test_bus_capacity_overflow: попытка посадить в автобус больше людей, чем позволяет capacity, выбрасывает ошибку переполнения. 4. test_ticket_double_validation: повторная попытка валидации билета приводит к TicketAlreadyUsedError. 5. test_route_minimum_stops: создание маршрута с 1 остановкой выбрасывает ValueError.
Запуск тестов в терминале:
pytest tests/ -v
Все тесты должны проходить с зеленым статусом (100% pass).

Блок 4. Контрольные вопросы для устной защиты (3 балла)

1. Почему для денежных расчетов в доменной модели используется Decimal, а не float? Продемонстрируйте проблему округления float на примере.
2. Как декоратор @property помогает реализовать сокрытие данных в Python?
3. Что произойдет, если разрешить внешнему коду напрямую изменять bus.current_passengers = -5? К каким бизнес-сбоям это приведет?
4. Чем исключение InsufficientFundsError лучше возврата булевого значения False при списании денег?

---

4. Чек-лист перед сдачей работы

- ☐ В коде отсутствуют заглушки pass, // TODO и print-отладка в бизнес-логике.
- ☐ Все публичные методы имеют аннотации типов (Type Hints) и docstrings.
- ☐ Все сущности защищают свои инварианты через исключения предметной области.
- ☐ Тесты pytest проходят без ошибок (100% passed).
- ☐ Код оформлен по стандартам PEP 8 (flake8 / ruff / black).